ProofGate

AI-compiled lending policies, enforced by Attestcoin proofs.

A lender writes rules in plain English. An AI compiles them into a policy committed on-chain BEFORE anyone applies. Credit moves only when the policy allows it AND the Attestcoin Protocol has cryptographically proven the repayment history.

The AI interprets the policy. The contract enforces it.

BUIDL CTC 2026 Fall - AI track

The problem: AI agents with money can’t be trusted

LLMs can FABRICATE repayment histories, INFLATE amounts, and ’reinterpret’ rules into what the lender never agreed to.

Contracts can’t judge free-text rules - that judgment is the one job only the AI can do. So keep the AI, but bind it:

1. to its OWN COMMITTED POLICY (immutable, on-chain, pre-application),
2. to CRYPTOGRAPHICALLY PROVEN cross-chain facts (Attestcoin Protocol).

Every release pays the decoded, proven total - never the AI’s claim.

Architecture

PolicyRegistry - English policy -> LLM-compiled struct + policyTextHash,
committed immutable. Versioning = new policyId.
ProofGate - submitRepaymentProof (verify via precompile -> replay guard ->
decode Transfer from the proven receipt -> policy checks), then
requestCredit (count >= min, claim <= cap, mint min(decoded, cap)).
TypeScript agent - llm.ts (provider-agnostic) -> compile_policy.ts ->
decide.ts. Ordinary key, ZERO privileged roles.

Contracts frozen at v1.0-contracts-frozen; 4/4 Foundry scenes run offline
on REAL Attestcoin proof fixtures.

Five scenes, all live on CC3 testnet

1 Honest (3x proven repayments) -> 50 PGC released   0x0fe91edf..7b33e1
2 Fabrication (tampered byte) -> PRECOMPILE REJECTED 0x1701337e..e60838
3 Inflation (claimed 500, proven 50) -> released 50   0x8c609657..90f3fb
4a Strict policy (needs 5, has 3) -> REVERTED   0xc89c97c2..81612
4b Agent beyond own cap (600 > 500) -> REVERTED   0xd2b53bf5..ede21
5 Live AI: English -> policy -> decision -> 50 PGC   0x4769f3a7..e7b6ff

Full audit trail: deployments.md - every hash a real testnet transaction.

Decision receipts: the AI vs the proof, forever

```
event CreditReleased(
  uint256 indexed policyId, address indexed borrower,
  uint256 releasedAmount,  // decoded from proofs - what moved
  uint256 claimedAmount,   // what the AI said
  bytes32 rationaleHash,   // keccak256 of the LLM’s own reasoning
  uint256 proofsUsed);
```

Scene 3 on-chain: released=50, claimed=500 - the inflation attempt is permanently recorded next to the truth. Any auditor can hash the rationale and compare it against the proven facts.

Attestcoin Protocol depth + resilience

22 verifyAndEmit calls across 7 committed policies (21 verified,
1 rejected - the fabrication scene). Full EVM receipts; event logs
decoded on-chain via the official EvmV1Decoder.
Synchronous in-tx verification; merkle + continuity proofs from the
Proof Builder; per-policy replay guards; multi-proof aggregation by default.

Resilience, proven live: an RPC ETIMEDOUT hit mid-demo. The agent is
resumable - it submits only proofs not yet on record (the replay guard
makes the boundary exact) - and finished the scene without manual repair.

What’s next

Mainnet policy templates + multi-source-chain histories (chainKey routing is already per-policy).
Richer compiled policies (rate limits, cooldowns) behind the same commit-then-enforce pattern.
Attested decision receipts: sign rationale + evidence bundle, anchor the signature next to rationaleHash.
Agent reputation: CreditReleased claimed-vs-released is a public, machine-readable honesty score for AI underwriters.

ProofGate: natural-language policy in, cryptographic proof out.

github.com/Pizzylee001/proofgate